Practical Exam

Series Overview

Exam	Application	Hooks	Points
1	Smart Task Counter	useState	20
2	Live Search and Timer	useEffect	20
3	Accessible Form Toolkit	useRef and useState	20
4	Application Preferences	useContext	20
5	Product Explorer Performance Repair	useMemo and useCallback	20
6	Issue Tracker State Engine	useReducer and custom Hooks	20

General Submission Rules

1. Submit the complete project folder without node_modules, together with a repository link when Git is required.
2. The application must run with npm install followed by npm run dev.
3. Do not install a state management library. Use only React Hooks for the required state logic.
4. Include a short README that identifies completed requirements, known limitations, and test steps.
5. During the demonstration, explain one bug encountered and the code change that fixed it.

Practical Exam 1 Smart Task Counter

Primary Hooks	Points	Difficulty
useState	20	Foundational

Scenario

Build a compact task counter for a study session. The application tracks completed tasks, the target number of tasks, and a status message derived from state.

Functional Requirements

Requirement	Evidence	Points
Create count and target state values with valid initial values.	Both values appear and update correctly.	4
Add Increment, Decrement, and Reset controls.	Count never becomes negative. Reset returns to zero.	5
Add a plus two control using two queued functional updates.	One click increases count by exactly two.	4
Display Not started, In progress, or Target reached.	Status changes from current state without separate status state.	4
Disable Increment and plus two after reaching the target.	Buttons respond correctly at the boundary.	3

Required Debugging Case

The following handler increases the value by only one. Repair it and explain why React processes both original calls from the same captured value.

```
const addTwo = () => {
  setCount(count + 1);
  setCount(count + 1);
};
```

Validation Cases

- Target 5 and count 0
- Count 4 followed by plus two
- Count 0 followed by Decrement
- Reset after the target is reached

Practical Exam 2 Live Search and Timer

Primary Hooks	Time	Points	Difficulty
useEffect and useState	45 minutes	20	Developing

Scenario

Build a searchable user list with a session timer. Use the supplied local data so the exam does not depend on internet access.

Starter Data

```
const USERS = [
  { id: 1, name: 'Ana Cruz', role: 'Developer' },
  { id: 2, name: 'Ben Lim', role: 'Designer' },
  { id: 3, name: 'Cara Yu', role: 'Tester' },
  { id: 4, name: 'Dan Go', role: 'Developer' }
];
```

Functional Requirements

Requirement	Evidence	Points
Filter users after the query changes.	A short effect delay simulates a search request.	4
Cancel the previous delayed search.	Fast typing does not display an outdated result.	4
Run a one second session timer.	Elapsed seconds increase once per second.	4
Clean up the timer on unmount.	No duplicate timer continues after navigation or remount.	4
Show the number of visible results in the document title.	The title synchronizes whenever the result count changes.	4

Required Debugging Cases

6. Repair an effect that depends on the same state value it updates and therefore runs repeatedly.
7. Repair a timer callback that captures the initial seconds value and remains stuck at one.
8. Explain the cleanup functions used for the delayed search and interval.

Practical Exam 3 Accessible Form Toolkit

Primary Hooks	Points	Difficulty
useRef and useState	20	Developing

Scenario

Build a registration form that guides the user to invalid fields while preserving render driven form values in state.

Functional Requirements

Requirement	Evidence	Points
Create name, email, and password inputs controlled by state.	Values remain visible and editable.	4
Create refs for the three input elements.	Each ref is attached through the ref attribute.	3
Validate required name, valid email structure, and eight character password.	All invalid fields receive readable messages.	5
Focus the first invalid field after submission.	Focus order is name, email, then password.	4
Track the previous submitted email without causing a render.	The previous value appears after the next successful submission.	4

Required Debugging Case

The starter code displays clicks.current in the interface, but the display does not update. Replace the ref with state for visible data and state why ref mutation does not schedule a render.

Restriction

Do not read or write ref.current during rendering except for predictable initialization. Access DOM refs in event handlers or effects.

Practical Exam 4 Application Preferences

Primary Hooks	Points	Difficulty
useContext and useState	20	Intermediate

Scenario

Build a small three level component tree that shares theme and font size preferences without passing preference props through every level.

Required Component Tree

```
App
  PreferencesProvider
    Dashboard
      Toolbar
        PreferencesPanel
      ArticlePreview
```

Functional Requirements

Requirement	Evidence	Points
Create one context and a provider component.	Consumers import the same context object.	4
Share theme, fontSize, and update functions.	Nested components access all required values.	4
Toggle light and dark themes.	Dashboard and ArticlePreview update together.	4
Change font size among 14, 16, and 18 pixels.	The preview changes without prop drilling.	4
Create usePreferences and reject use outside the provider.	The custom access Hook returns context or throws a clear error.	4

Required Debugging Cases

9. Repair a consumer that receives only the default value because no provider wraps the tree.
10. Repair a project where the provider and consumer import two separately created context objects.
11. Explain why moving shared state into context removes intermediate preference props.

Practical Exam 5 Product Explorer Performance Repair

Primary Hooks	Time	Points	Difficulty
useMemo and useCallback	50 minutes	20	Intermediate

Scenario

Repair a product explorer that performs an expensive filter and sends a new callback to a memoized child during every parent render.

Starter Constraints

- Use at least 500 generated product records.
- Provide text search, category selection, and price sorting.
- Wrap ProductList with React.memo.
- Include an unrelated counter in the parent to test unnecessary work.

Functional Requirements

Requirement	Evidence	Points
Memoize the filtered and sorted product list.	The calculation reruns only when its true inputs change.	5
Memoize the item selection callback.	The callback stays stable until a required dependency changes.	4
Prevent ProductList from rerendering for the unrelated counter.	Console evidence shows stable child rendering.	4
Keep all dependency arrays complete.	Search, category, sort order, and selected behavior stay current.	4
Document one case where memoization would be unnecessary.	README gives a technically accurate example.	3

Required Debugging Cases

12. Add a missing query dependency to a useMemo calculation that displays stale results.
13. Add a missing project or selected value dependency to a useCallback that uses outdated data.
14. Move an analytics call out of useMemo and into useEffect.

Practical Exam 6 Issue Tracker State Engine

Primary Hooks	Time	Points	Difficulty
useReducer and custom Hooks	60 minutes	20	Advanced

Scenario

Build an issue tracker whose state transitions are handled by a pure reducer. Extract persistence into a custom Hook.

Required State Shape

```
{
  issues: [],
  filter: 'all',
  nextId: 1
}
```

Required Actions

```
ADD_ISSUE      payload: { title }
TOGGLE_ISSUE   payload: { id }
DELETE_ISSUE   payload: { id }
SET_FILTER     payload: { filter }
CLEAR_DONE
```

Functional Requirements

Requirement	Evidence	Points
Implement all five reducer actions.	Every transition returns the expected next state.	5
Keep the reducer pure and immutable.	No existing state object or issue object is mutated.	4
Preserve state for unknown actions.	The default branch returns the existing state.	2
Create usePersistentReducer using localStorage.	State loads once and saves after state changes.	4
Show All, Open, and Done issue views.	Filtering works and the counts remain accurate.	3
Add three reducer tests using sample states and actions.	Tests cover add, toggle, and clear completed behavior.	2

Required Debugging Cases

15. Replace direct state mutation with a new state object and new issue objects where changes occur.
16. Repair a mismatched action shape in which the reducer reads action.payload but dispatch sends action.title.
17. Repair a default branch that returns undefined.

Criterion	Full Credit	Partial Credit	Points
Functional correctness	All required behaviors and boundary cases work.	Some behaviors work, but a required case fails.	8
Hook selection and usage	Hooks match their purpose and follow call order rules.	The application works with avoidable misuse or incomplete dependencies.	5
Debugging evidence	Required faults are repaired and accurately explained.	Repairs work but explanations are incomplete.	3
Code quality	Components, names, state shape, and logic are clear.	Code is understandable but duplicated or difficult to trace.	2
Demonstration and testing	Student demonstrates required cases and explains one decision.	Demonstration misses a required case or explanation.	2